



Planejamento de Artigo Técnico

Título Sugerido: "Como reduzi em 50% os custos de Kubernetes ajustando o 'Invisível': O Guia de Sobrevivência de FinOps." **Subtítulo:** "Kafka em Dev, Agentes sem limites e PDBs travados: a anatomia de um cluster desperdiçador." **Público-Alvo:** DevOps, SREs, Tech Leads e CTOs.

1. Introdução: O Problema do "Padrão de Fábrica"

- **Gancho:** Comece falando sobre como é fácil subir um cluster EKS e instalar ferramentas (Helm install), mas como é difícil perceber o custo oculto das configurações padrão (*defaults*).
- **Contexto:** Descreva o cenário: Ambientes de Dev/HML rodando com configurações de Alta Disponibilidade (HA) idênticas à Produção.
- **O Sintoma:** "Nossos nós estavam com 20% de uso real de CPU, mas a AWS cobrava por máquinas inteiras. O Autoscaler não desligava nada. Por quê?"

2. Passo 1: Caçando os "Vizinhos Barulhentos" (Agentes de Infra)

- **A Investigação:** Mostre como identificamos que não eram as aplicações de negócio que consumiam recursos, mas sim os agentes de suporte (Datadog, Security, Logs).

O Comando Mágico: Compartilhe o comando para achar os vilões.

Bash

```
kubectl top pods -A --sort-by=cpu | head -10
```

-

- **O Erro Comum:** Explicar que muitos Helm Charts vêm sem `limits` ou com `requests` altos demais.

A Solução (Exemplo Prático): Mostre como "enjaulamos" o Datadog e o GuardDuty. Explique que em Dev, monitoramento não pode roubar CPU de negócio.

Bash

```
# O "antes" (Best Effort - Perigoso)
```

```
Requests: <vazio>
```

```
Limits: <vazio>
```

```
# O "depois" (Enjaulado)
```

```
kubectl set resources ds -n datadog datadog-agent -c agent \
```

```
--requests=cpu=100m,memory=256Mi \
```

```
--limits=cpu=500m,memory=512Mi
```

-

3. Passo 2: O Mito da Alta Disponibilidade em DEV

- **Conceito:** "Dev precisa de HA?" Questione a necessidade de manter clusters Kafka com 3 Brokers e Zookeepers replicados em ambiente de desenvolvimento.
- **O Impacto:** Mostre o custo de manter 3 JVMs pesadas rodando para processar mensagens de teste.

A Solução (Exemplo Prático): Como transformamos o Kafka em *Single-Node* via patch, economizando gigabytes de RAM instantaneamente.

Bash

Reduzindo a redundância desnecessária

```
kubectl patch kafkanodepool broker -n app-integrations \
--type='merge' -p '{"spec":{"replicas": 1}}'
```

- **Dica de Ouro:** Mencione a limpeza de PVCs órfãos (`kubectl delete pvc...`) para não pagar por discos EBS parados.

4. Passo 3: O Inimigo Silencioso (Pod Disruption Budgets)

- **O Mistério:** "Ajustamos tudo, os nós ficaram vazios, mas o Cluster Autoscaler **não** desligava as máquinas. O que houve?"
- **A Revelação:** Explique o conceito de **Deadlock de PDB**.
 - Se você tem `replicas: 1` e `minAvailable: 1`, o Kubernetes jamais deixará aquele nó ser drenado. Ele se torna um "refém".

O Diagnóstico:

Bash

```
kubectl get pdb -A
```

Procure por "ALLOWED DISRUPTIONS: 0"

-

A Solução: O script de limpeza para ambientes não-produtivos.

Bash

A "Vassoura" de PDBs para Dev/HML

```
kubectl get pdb -A --no-headers | grep -v "kube-system" | \
awk '{print "kubectl delete pdb " $2 " -n " $1}' | sh
```

-

5. O Grande Final: A Redução Física (Scale Down)

- **Resultado Visual:** Descreva a sensação de ver o `kubectl top nodes` mostrando nós vazios.
- **A Matemática da Economia:**
 1. Antes: 3 Nós (ou 4) subutilizados.
 2. Depois: 2 Nós na "Zona Verde" (60% de uso de memória).
 3. **Economia:** Redução de ~33% a 50% na fatura de EC2 para esses ambientes.
- **Checklist Final:**
 1. Right-sizing de Agentes.
 2. Remoção de HA em Dev/HML (Kafka/Ingress = 1 réplica).
 3. Limpeza de PDBs bloqueantes.
 4. Ajuste do ASG (Auto Scaling Group).

6. Conclusão e Lições Aprendidas

- Não confie nos *defaults* do Helm.
- Trate Dev e HML como ambientes efêmeros e baratos, não como réplicas de Produção.
- Observabilidade (saber quem consome CPU) é o primeiro passo para FinOps.



Dicas Visuais para o seu Blog

- **Screenshots:** Tire prints do seu terminal mostrando o "Antes" (lista de pods cheia, nós com pouca % de uso) e o "Depois".
- **Diagrama:** - Um gráfico simples mostrando "Nós Antes (3)" vs "Nós Depois (2)" e os componentes removidos.

Sugestão de Call to Action (CTA)

"E você, já verificou se seus agentes de monitoramento estão custando mais que sua aplicação? Comente abaixo qual foi o maior desperdício que você já achou no seu cluster."